

Amazon OpenSearch Service

Full-Text Search · Product Catalog · Autocomplete · Facets · UltraWarm · ShopFlow

AWS Tutorial · ShopFlow

12-01

What is Amazon OpenSearch Service?

■ Amazon OpenSearch Service

OpenSearch Service is a managed Elasticsearch/OpenSearch cluster on AWS. It stores documents as JSON, indexes every field, and enables full-text search, filters, aggregations, and analytics in milliseconds — things that are slow or impossible with SQL databases. It is the backbone of ShopFlow product search.

Example: ShopFlow product database in Aurora has 2 million products. A SQL query LIKE "%red running shoes%" scans every row — slow and CPU-intensive. OpenSearch indexes every word in every product name, description, and category. A search returns matching products in < 50ms regardless of catalog size.

Use Case	Aurora PostgreSQL	OpenSearch
Find product by ID	SELECT * FROM products WHERE id=123 Not made (primary key)	Not made (primary key)
Full-text search across name+description	LIKE "%red shoes%" — full table scan, slow	Multi-field full-text search in < 50ms
Autocomplete as user types	Not practical — too slow on 2M rows	Completion suggester — sub-10ms response
Filter: price < \$50 AND brand=Nike	Possible but without perfect indexes	Fast boolean filter on pre-indexed numeric/keyword fields
Aggregations: count per brand/category	GROUP BY — possible but expensive under Aggs	Aggregations (facets) in same query as search
Fuzzy match (typos: "Nikee" -> "Nike")	No built-in fuzzy matching	fuzziness=AUTO — handles 1-2 character typos

12-02

OpenSearch vs Elasticsearch — Background

OpenSearch is a fork of Elasticsearch 7.10, created in 2021 when Elastic changed its license. AWS OpenSearch Service supports both OpenSearch and Elasticsearch APIs. For new projects, use OpenSearch. For migration from Elasticsearch, the API is compatible.

Aspect	Amazon OpenSearch Service	Self-managed Elasticsearch
Management	Fully managed: OS patching, backups, scaling	Manually manage EC2, OS, Elasticsearch upgrades, backups
High availability	Multi-AZ with automatic failover; replica shards	Manually configure replicas across AZs yourself
Snapshots	Automated daily snapshots; manual snapshots	Configure snapshot lifecycle policy manually
Security	VPC, fine-grained access control, encryption at rest	Configure security plugin, TLS certificates, user auth manually
Scaling	Add/remove data nodes; UltraWarm for cost-effective scaling	Manually scale; risk of data loss if not careful
Cost	\$0.24/hr per r6g.large data node (standard); UltraWarm \$0.024/hr	EC2 cost \$0.024/hr + engineering time
ShopFlow choice	Managed OpenSearch — 2 data nodes, 2 AZs, r6g.large instance	

12-03

Create an OpenSearch Domain — Console Walkthrough

An OpenSearch Domain is a managed cluster. For ShopFlow, we create a 2-node domain in 2 AZs for high availability. The domain will hold the product search index.

■ Create OpenSearch Domain

1. AWS Console -> search "OpenSearch" -> click Amazon OpenSearch Service
2. Click "Create domain"
3. Domain name: shopflow-search
4. Deployment type: Production (enables multi-AZ)
5. Availability Zones: 2-AZ
6. Version: OpenSearch 2.11 (latest)
7. Instance type: r6g.large.search (Graviton, 2 vCPU, 16 GB RAM) — good starting point
8. Number of nodes: 2 (1 per AZ)
9. Storage per node: 100 GB gp3 EBS
10. Network: VPC access (NOT public — for production, always VPC)
11. VPC: shopflow-vpc, Subnets: data-subnet-1a, data-subnet-1b
12. Fine-grained access control: Enable, create master user (store in Secrets Manager)
13. Encryption: Enable at rest and in transit (TLS)
14. Click "Create" — takes 10-15 minutes

■ aws > OpenSearch > Domains > shopflow-search

Domain overview

■ shopflow-search

- Endpoint: <https://search-shopflow-search-xyz.us-east-1.es.amazonaws.com>
- Status: Active | Version: OpenSearch_2.11 | AZs: 2
- Data nodes: 2 x r6g.large (2 vCPU, 16 GB RAM, 100 GB gp3 each)
- Total storage: 200 GB | Indices: 0 | Documents: 0
- Network: VPC (shopflow-vpc) | Encryption: at-rest + in-transit
- Domain is active and accepting connections from within VPC

12-04

OpenSearch Domain with CDK

CDK creates the OpenSearch domain with all production settings: VPC placement, encryption, fine-grained access control, and automated snapshots.

```
// CDK: ShopFlow OpenSearch Domain Domain searchDomain = Domain.Builder.create(this,
"ShopFlowSearch") .domainName("shopflow-search") .version(EngineVersion.OPENSEARCH_2_11)
.capacity(CapacityConfig.builder() .dataNodeInstanceType("r6g.large.search") // Graviton –
better price/perf .dataNodes(2) // 2 nodes across 2 AZs .build()) .ebs(EbsOptions.builder()
.enabled(true) .volumeType(EbsDeviceVolumeType.GP3) .volumeSize(100) // 100 GB per node (200 GB
total) .iops(3000) // gp3 baseline .throughput(250) // MB/s .build()) .vpc(vpc)
.vpcSubnets(List.of(SubnetSelection.builder() .subnets(List.of(dataSubnet1a, dataSubnet1b))
.build())) .securityGroups(List.of(searchSg)) .zoneAwareness(ZoneAwarenessConfig.builder()
.enabled(true) .availabilityZoneCount(2) .build())
.encryptionAtRest(EncryptionAtRestOptions.builder().enabled(true).build())
.nodeToNodeEncryption(true) .enforceHttps(true)
```

```
.fineGrainedAccessControl(AdvancedSecurityOptions.builder() .masterUserName("shopflow-admin")
.masterUserPassword(masterPassword) .build()) .automatedSnapshotStartHour(3) // daily snapshot
at 3 AM UTC .logging(LoggingOptions.builder() .slowSearchLogEnabled(true) // log queries > 5s
.slowIndexLogEnabled(true) // log indexing > 500ms .appLogEnabled(true) .build()) .build();
```

12-05

Index Design — ShopFlow Product Search Index

Index design is the most important OpenSearch decision — done wrong, search quality suffers and queries are slow. ShopFlow products index stores one document per product with carefully typed fields.

```
// Create ShopFlow products index with explicit field mappings // (using curl or AWS SDK;
typically done during application startup or migration) PUT /products { "settings": {
"number_of_shards": 2, // 1 shard per data node "number_of_replicas": 1, // 1 replica per shard
(stored on other node) "analysis": { "analyzer": { "shopflow_standard": { // custom analyzer for
product names "type": "custom", "tokenizer": "standard", "filter": ["lowercase", "stop",
"snowball"] // "running" -> "run" } } }, "mappings": { "properties": { "productId": { "type":
"keyword" }, // exact match only "name": { "type": "text", "analyzer": "shopflow_standard",
"fields": { "keyword": { "type": "keyword" } } }, "description": { "type": "text", "analyzer":
"shopflow_standard" }, "brand": { "type": "keyword" }, // exact: "Nike" not "nike running"
"category": { "type": "keyword" }, // "Footwear > Running" "price": { "type": "double" }, //
numeric range filter "inStock": { "type": "boolean" }, "rating": { "type": "float" },
"reviewCount": { "type": "integer" }, "tags": { "type": "keyword" }, // ["waterproof","trail"]
"imageUrl": { "type": "keyword", "index": false }, // stored, not searched "updatedAt": {
"type": "date" } } } }
```

Field Type	Use When	ShopFlow Examples
text	Full-text search needed; field is analyzed (tokenized)	description — users search within these
keyword	Exact match, filtering, aggregations, sorting	brand, category, productId — "filter by Nike" not "search within Nike"
double/float/integer	Numeric range queries and sorting	price (range \$20-\$50), rating (sort by rating), reviewCount
boolean	Yes/no filter	inStock=true filter; shippingIncluded=true
date	Date range queries and time-based sorting	updatedAt for "new arrivals in last 7 days"

12-06

Indexing Products — Java 21 OpenSearch Client

Indexing adds or updates documents in OpenSearch. ShopFlow indexes products from Aurora via the DynamoDB Streams -> Lambda -> OpenSearch pipeline (section 09-07). Each product in Aurora is also synced to OpenSearch on create/update.

```
// Java 21: Index a product to OpenSearch // Using opensearch-java client (version 2.x)
OpenSearchClient client = new OpenSearchClient( new RestClientTransport(
RestClient.builder(HttpHost.create( System.getenv("OPENSEARCH_ENDPOINT"))))
.setHttpClientConfigCallback(b -> b .addInterceptorFirst(new
AwsRequestSigningApacheInterceptor( "es", signerV4, credentialsProvider, "us-east-1"))))
.build(), new JacksonJsonpMapper()); // Index a product document ProductDocument doc =
ProductDocument.builder() .productId("SKU-12345") .name("Nike Air Max 270 Running Shoe")
.description("Lightweight running shoe with Max Air cushioning") .brand("Nike")
.category("Footwear > Running") .price(149.99) .inStock(true) .rating(4.6f) .reviewCount(2847)
.tags(List.of("running", "cushioned", "lightweight")) .updatedAt(Instant.now()) .build();
client.index(IndexRequest.of(req -> req .index("products") .id(doc.productId()) // Aurora
product ID as OpenSearch document ID .document(doc))); // Bulk indexing (more efficient for
large batches) BulkRequest.Builder bulkReq = new BulkRequest.Builder(); for (ProductDocument p
: products) { bulkReq.operations(op -> op .index(idx ->
idx.index("products").id(p.productId()).document(p))); } client.bulk(bulkReq.build());
```

12-07

Search Queries — Multi-Field, Filters, and Facets

ShopFlow product search combines full-text search (what the user typed) with boolean filters (in-stock, price range, brand) and returns aggregations (facets: how many results per brand/category). This is a bool query with must + filter + aggregations.

```
// Java 21: Full product search query SearchResponse response = client.search(
SearchRequest.of(req -> req .index("products") .query(q -> q .bool(b -> b // must: full-text
search in name + description (scored) .must(m -> m .multiMatch(mm -> mm .query("red running
shoes") .fields(List.of("name^3", "description^1")) // name 3x weight .fuzziness("AUTO") //
handle typos .type(TextQueryType.BestFields))) // filter: boolean filters (no scoring impact,
cached) .filter(f -> f .term(t -> t.field("inStock").value(true))) .filter(f -> f .range(r ->
r.field("price") .gte(JsonData.of(20)).lte(JsonData.of(100)))) .filter(f -> f .terms(t ->
t.field("brand") .terms(tv -> tv.value(List.of(FieldValue.of("Nike"),
FieldValue.of("Adidas")))))) // aggregations: return facets alongside results
.aggregations(Map.of( "brands", Aggregation.of(a -> a .terms(t -> t.field("brand").size(20))),
"categories", Aggregation.of(a -> a .terms(t -> t.field("category").size(10))), "price_ranges",
Aggregation.of(a -> a .range(r -> r.field("price") .ranges(List.of( AggregationRange.of(ar ->
ar.to(50.0)), AggregationRange.of(ar -> ar.from(50.0).to(100.0)), AggregationRange.of(ar ->
ar.from(100.0)))))) .size(20).from(0) // page 1: 20 results .sort(s -> s.field(f ->
f.field("_score").order(SortOrder.Desc))), ProductDocument.class);
```

12-08

Autocomplete — Instant Search Suggestions

ShopFlow shows product name suggestions as the user types in the search box. OpenSearch completion suggester provides sub-10ms autocomplete by storing suggestions in a prefix-indexed in-memory data structure.

```
// Index mapping: add completion field to products index "mappings": { "properties": {
"suggest": { "type": "completion", "analyzer": "shopflow_standard" } } } // When indexing a
product, populate the suggest field: ProductDocument doc = ProductDocument.builder()
.name("Nike Air Max 270 Running Shoe") .suggest(CompletionField.builder() .input(List.of( "Nike
Air Max 270", // exact product name prefix "Air Max 270", // without brand prefix "Nike Running
Shoe", // generic match "Running Shoes")) // category suggestion .weight(doc.reviewCount()) //
popular products rank higher .build()) .build(); // Query: get autocomplete suggestions for
partial input "nike ai" SuggestResponse resp = client.search( SearchRequest.of(req -> req
.suggest(sug -> sug .suggesters(Map.of( "product-suggest", FieldSuggester.of(fs -> fs
.prefix("nike ai") .completion(cs -> cs .field("suggest") .size(5) // return top 5 suggestions
.skipDuplicates(true) .fuzzy(SuggestFuzziness.of(sf -> sf .fuzziness("1")))))) // allow 1
typo .size(0)), // no search results – just suggestions ProductDocument.class);
```

12-09

UltraWarm and Cold Storage — Cost-Effective Old Data

Not all indexed data is queried equally. Last week product search logs are queried frequently; last year logs are rarely accessed. OpenSearch UltraWarm stores less-accessed data at \$0.024/GB (vs \$0.135/GB for hot storage) while keeping it searchable.

Storage Tier	Cost/GB/month	Access Latency	Use Case	ShopFlow Data
Hot (EBS)	\$0.135	< 100ms	Frequently accessed; recent data	Current product catalog (200 GB)
UltraWarm (S3-backed)	\$0.024 (82% cheaper)	100-500ms	Accessed weekly or less	Product search logs older than 30 days
Cold (S3)	\$0.01 (93% cheaper)	Minutes (manual attach)	Archived; compliance; rarely accessed	Product analytics older than 1 year

```
// CDK: Enable UltraWarm on OpenSearch domain Domain searchDomain = Domain.Builder.create(this,
"ShopFlowSearch") .capacity(CapacityConfig.builder() .dataNodeInstanceType("r6g.large.search")
.dataNodes(2) .warmInstanceType("ultrawarm1.medium.search") .warmNodes(2) // 2 UltraWarm nodes
for warm data .build()) .build(); // Index State Management (ISM) policy: auto-move old indices
to UltraWarm // Create via OpenSearch Dashboards or API: // After 30 days: migrate index from
hot to UltraWarm // After 90 days: migrate to cold storage // After 365 days: delete
```

12-10

OpenSearch Dashboards — Visualize and Monitor

OpenSearch Dashboards (formerly Kibana) is a browser-based UI for visualizing data, creating dashboards, running ad-hoc queries, and managing index settings. It is included with every OpenSearch Service domain.

■ Access OpenSearch Dashboards

1. OpenSearch Console -> select shopflow-search domain
2. Click the "OpenSearch Dashboards URL" (e.g. https://search-shopflow-search-xyz.es.amazonaws.com/_dashboards)
3. Note: domain is VPC-only; access requires VPN or SSH tunnel through bastion
4. Tunnel command: `ssh -L 9200:shopflow-search-endpoint:443 ec2-user@bastion-ip`
5. Then open: https://localhost:9200/_dashboards in browser
6. Login with master user credentials (stored in Secrets Manager)
7. Discover tab: explore all indexed documents; test queries interactively
8. Dev Tools tab: run raw OpenSearch API queries (like curl but in browser)
9. Dev Tools example: `GET /products/_count` -> verify 2,000,000 products indexed
10. Dev Tools example: `POST /products/_search` with query body to test search behavior

INFO Access Control for Dashboards

ShopFlow OpenSearch is VPC-only for security. Engineers access Dashboards via an SSH tunnel through the bastion host (or AWS Systems Manager Session Manager port forwarding — no bastion needed). For a customer-facing analytics dashboard, use a Lambda function as a proxy: Lambda calls OpenSearch API inside VPC and returns results to the browser.

12-11

Search Relevance Tuning — ShopFlow Product Search

Default OpenSearch scoring (BM25) ranks by term frequency and rarity. For e-commerce, you also want to boost popular products, recently added items, and exact matches. Field boosting and function score queries let you blend text relevance with business signals.

```
// Function score query: blend text relevance with business signals SearchRequest.of(req -> req
.query(q -> q .functionScore(fs -> fs .query(textQuery) // base: full-text search query
.functions(List.of( // Boost in-stock products 2x over out-of-stock FunctionScore.of(f -> f
.filter(Filter.of(fi -> fi .term(t -> t.field("inStock").value(true)))) .weight(2.0)), // Boost
by rating (0-5 -> multiply score by 0-1) FunctionScore.of(f -> f .fieldValueFactor(fvf -> fvf
.field("rating") .factor(0.2) // 5-star rating adds 1.0 to score .missing(2.5))), // default
for missing rating // Boost by log of review count (popular items rank higher)
FunctionScore.of(f -> f .fieldValueFactor(fvf -> fvf .field("reviewCount")
.modifier(FieldValueFactorModifier.Log1p) .factor(0.5)))) .scoreMode(FunctionScoreMode.Sum)
.boostMode(FunctionBoostMode.Multiply))) .sort(SortOptions.of(so -> so .score(ScoreSort.of(ss
-> ss.order(SortOrder.Desc))))))
```

12-12

Monitoring OpenSearch — Key Metrics and Alarms

OpenSearch Service publishes metrics to CloudWatch. The critical metrics to monitor: cluster health (red = data loss risk), JVM memory pressure, and search latency.

Metric	Alarm Threshold	Meaning and Action
ClusterStatus.red	> 0 for 1 minute	PRIMARY SHARDS MISSING — data loss risk. Immediate action: check
ClusterStatus.yellow	> 0 for 10 minutes	Replica shards unassigned (no data loss, but no HA). Check if a node is d
JVMMemoryPressure	> 85% for 15 minutes	GC pressure; performance degrades above 85%. Add data nodes or incre
FreeStorageSpace	< 20 GB per node	Less than 20 GB free. OpenSearch stops indexing at 5 GB free. Add EBS
SearchLatencyP99	> 500ms for 5 minutes	Search response time degrading. Check JVM pressure, slow log, query co

Product search data flows from two sources: the product catalog (Aurora -> Lambda -> OpenSearch on create/update) and order data for popularity signals (DynamoDB -> Lambda -> OpenSearch for reindex).

ShopFlow OpenSearch Architecture

Data Ingestion (Write Path)

Aurora products table -> DynamoDB Streams -> SearchIndexLambda -> OpenSearch

product-svc REST API -> direct OpenSearch index on product create/update

Nightly batch: full reindex from Aurora to rebuild relevance signals

OpenSearch Cluster (shopflow-search)

2 data nodes (r6g.large, 100 GB gp3 each) across 2 AZs

products index: 2 primary shards + 2 replica shards

2 UltraWarm nodes: search logs older than 30 days

Search Query Path

Browser -> API Gateway HTTP API -> ProductSearchLambda (VPC)

ProductSearchLambda -> OpenSearch bool query (text + filters + aggs)

OpenSearch returns: 20 products + facets + total count in < 50ms

Autocomplete Path

Browser keypress -> API Gateway -> AutocompleteLambda -> completion suggest

Completion suggester returns 5 suggestions in < 10ms

Create an OpenSearch domain, index sample products, and run search queries. Time estimate: 90 minutes (most time is domain creation: 10-15 minutes).

■ Lab: OpenSearch Product Search

1. Create OpenSearch domain: shopflow-search-lab (Development mode, 1 node, t3.small.search)
2. (Development: single node, no VPC, public access — fine for lab, not production)
3. Wait 10-15 minutes for domain to become Active
4. Open OpenSearch Dashboards URL from domain overview
5. Dev Tools -> run: PUT /products with index mapping from section 12-05
6. Dev Tools -> run: POST /products/_bulk with 5 sample product documents
7. Dev Tools -> run: GET /products/_count -> verify 5 documents indexed
8. Dev Tools -> run: POST /products/_search with query: {"query":{"match":{"name":"shoes"}}}
9. Verify results include matching products
10. Add filter: {"query":{"bool":{"must":{"match":{"name":"shoes"},"filter":{"term":{"inStock":true}}}}}}
11. Add aggregation: add "aggs":{"brands":{"terms":{"field":"brand"}}} to see brand counts
12. Cleanup: delete domain (to avoid \$0.36/hr charges for t3.small)

12-15

Common Questions and Best Practices

Question	Answer
My OpenSearch cluster turned red. What does that mean?	What does that mean? More primary shards are unassigned (not just replicas). Run GET /_cluster/health?v
Should I keep Aurora AND OpenSearch in my app?	Yes in Aurora. Aurora is the source of truth; OpenSearch is the search index. Never store data only in Op
How do I handle index mapping changes?	OpenSearch does not allow changing existing field types. Create a new index with the updated ma

Best Practice	Anti-Pattern to Avoid
Use field aliases ("products_alias") so you can swap underlying indices in application code	Directly updating indices in application code forces code deployment for every re
Set number_of_shards at index creation; cannot change later	Using default shard count (1) for a large index — causes shard hotspots; canr
Monitor JVM memory pressure; alarm at 85%	Ignoring JVM pressure until queries start timing out at 95%+ memory pressure
Use keyword type for exact-match fields (brand, category, etc.)	Using text type for brand — "Nike" tokenizes to ["nike"], breaking exact filter
Enable slow logs for queries > 500ms; analyze weekly	For slow log optimization to identify and optimize expensive queries

12-16

Index State Management — Automate Index Lifecycle

Index State Management (ISM) automates time-based index operations: moving old indices to UltraWarm, deleting stale logs, and triggering rollups. Without ISM, your hot storage fills up as indices accumulate and performance degrades.

```
// ISM Policy: automate index lifecycle for shopflow-search-logs // Create via OpenSearch Dashboards -> Index Management -> ISM Policies -> Create { "policy": { "description": "ShopFlow search log lifecycle", "states": [ { "name": "hot", "actions": [ { "rollover": { "min_doc_count": 100000 } } ], "transitions": [ { "state_name": "warm", "conditions": { "min_index_age": "30d" } } ] }, { "name": "warm", "actions": [ { "warm_migration": { } } ], "transitions": [ { "state_name": "cold", "conditions": { "min_index_age": "90d" } } ] }, { "name": "cold", "actions": [ { "cold_migration": { "timestamp_field": "@timestamp" } } ], "transitions": [ { "state_name": "delete", "conditions": { "min_index_age": "365d" } } ] }, { "name": "delete", "actions": [ { "delete": { } } ], "transitions": [ ] } ] } }
```


ISM State	Trigger	Action	ShopFlow Setting
hot -> warm	Index age > 30 days	Move to UltraWarm (S3-backed, 82% cheaper)	Search logs: 30 days hot, then warm
warm -> cold	Index age > 90 days	Move to cold storage (detached S3)	Search logs: 90 days warm, then cold
cold -> delete	Index age > 365 days	Permanently delete index	After 1 year, logs purged
hot rollover	> 100K docs or > 50 GB	Create new index, point alias to new index	Prevents oversized shards

12-17

Search Quality Testing — Precision and Recall

Good search is not just about speed — it is about relevance. Users who get irrelevant results do not buy. ShopFlow measures search quality with Precision@K (are the top results relevant?) and MRR (is the most relevant result near the top?).

Metric	Formula	ShopFlow Target	How to Measure
Precision@5	Relevant results in top 5 / 5	> 80%: 4+ of top 5 results are relevant	Manual judgment: give 50 test queries to 3 evaluators
Recall@20	Relevant results found in top 20 / total relevant	> 70%: relevant results found in most queries	Requires knowing ground truth; use past purchase history
MRR (Mean Reciprocal Rank)	Average of 1/rank of first relevant result	> 0.75: first relevant result is usually in top 3	Position of first product the user clicked from search results
Click-through rate	Clicks / searches	> 60%: most searches result in clicks	CloudFront access logs + front-end analytics
Zero-result rate	Queries with 0 hits / all queries	< 5%: rarely should return no results	Log search queries + result counts; alert if > 5%

TIP A/B Test Search Algorithms

Run two search query variations simultaneously: 50% of users get the current relevance algorithm, 50% get the new one. Measure click-through rate and conversion rate for each group using CloudWatch metrics from the front-end. This is how ShopFlow discovered that adding reviewCount as a ranking signal increased conversion rate by 8%.

12-18

Vector Search — AI-Powered Product Recommendations

OpenSearch k-NN (k-nearest neighbor) stores vector embeddings and finds semantically similar documents. ShopFlow uses this for "similar products" recommendations: convert each product description to a vector using Amazon Bedrock Titan Embeddings, then find products with similar vectors.

```
// Index mapping: add vector field for product embeddings PUT /products { "settings": {
  "index.knn": true, "index.knn.space_type": "cosinesimil" }, "mappings": { "properties": {
    "embedding": { "type": "knn_vector", "dimension": 1536, // Titan Embeddings v2 dimension
    "method": { "name": "hnsw", // Hierarchical Navigable Small World graph "engine": "nmslib",
    "parameters": { "m": 16, "ef_construction": 128 } } } } } } // Java: Generate embedding + index
product BedrockRuntimeClient bedrock = BedrockRuntimeClient.create(); InvokeModelResponse
embeddingResponse = bedrock.invokeModel( InvokeModelRequest.builder()
.modelId("amazon.titan-embed-text-v2:0") .body(SdkBytes.fromUtf8String( "{"inputText": "" +
product.name() + " " + product.description() + "}") ) .build()); float[] embedding =
parseEmbedding(embeddingResponse.body().asUtf8String()); // k-NN search: find 5 most similar
products to a given product SearchRequest.of(req -> req .knn(List.of(KnnQuery.of(knn -> knn
.field("embedding") .vector(targetEmbedding) .k(5)))) // return 5 nearest neighbors .size(5))
```

12-19

Cross-Cluster Search — Search Multiple Domains

As ShopFlow grows, different domains handle different data: product catalog in one domain, order history in another, support tickets in a third. Cross-Cluster Search lets you query multiple OpenSearch domains in a single request.

Aspect	Single Domain	Cross-Cluster Search
Use case	All data in one domain (ShopFlow current state)	Multiple specialized domains; query all at once
Setup	No setup needed	Configure connections: source domain lists remote domain endpoints

Aspect	Single Domain	Cross-Cluster Search
Query syntax	Standard: GET /products/_search	Prefix remote: GET /remote-orders:orders/_search
Performance	Single-domain latency	Parallel query to all clusters; latency = slowest cluster
Cost	Standard domain pricing	Standard pricing per domain; no extra charge for cross-cluster
ShopFlow future	Product catalog domain	Product + orders + support tickets cross-cluster search

INFO When ShopFlow Will Need Cross-Cluster Search

Current scale: 2M products, all in one domain. At 50M products + order history search + customer support ticket search, a single domain becomes a management burden. At that scale, split into: products domain (read-heavy, optimized for fast search), orders domain (write-heavy, time-series), support domain (text-heavy, smaller size). Cross-cluster search unifies them for admin dashboards.

12-20

Reindexing — Zero-Downtime Schema Changes

Reindexing copies data from one index to another, allowing you to change mappings, add fields, or change analyzers without downtime. The key is using index aliases so the application never uses an index name directly.

■ Zero-Downtime Reindex with Aliases

1. Current state: application writes to "products" alias pointing to "products-v1"
2. Goal: change product name analyzer to custom shopflow_standard
3. Step 1: Create new index "products-v2" with updated mapping
4. Step 2: Start reindex: POST /_reindex with source=products-v1, dest=products-v2
5. Step 3: Monitor progress: GET /_tasks?action=*reindex (takes ~10 min for 2M docs)
6. Step 4: Wait for reindex to complete (compare doc counts: GET /products-v2/_count)
7. Step 5: Index any documents created during reindex: use updatedAt > reindex_start_time filter
8. Step 6: Atomic alias swap: POST /_aliases with remove products-v1 + add products-v2
9. Application now reads/writes products-v2 via "products" alias — zero downtime
10. Keep products-v1 for 24 hours as rollback option, then delete

12-21

OpenSearch Security — Fine-Grained Access Control

Fine-Grained Access Control (FGAC) in OpenSearch lets you restrict access per index, per document, or even per field. ShopFlow uses this to prevent the product-svc from reading the orders index.

```
// OpenSearch Dashboards -> Security -> Roles -> Create role // Role: product-svc-role {
"cluster_permissions": ["cluster:monitor/*"], "index_permissions": [{ "index_patterns":
["products*"], // only products indices "allowed_actions": [ "read", "search", "get",
"indices:data/write/index", // can write (index products) "indices:data/write/bulk" // can bulk
index ] // No "delete" permission – cannot delete the index }] // No permissions on orders*,
support* indices } // Map IAM role to OpenSearch role // Backend roles:
arn:aws:iam::123456789:role/shopflow-product-svc-task-role // -> maps to OpenSearch role:
product-svc-role // CDK: grant OpenSearch access to ECS task role
searchDomain.grantReadWrite(productSvcTaskRole); // Above creates resource-based policy;
combine with FGAC for fine-grained control
```

Access Pattern	FGAC Config	ShopFlow Use Case
Per-index	Grant read on products* only	product-svc cannot read orders index

Access Pattern	FGAC Config	ShopFlow Use Case
Per-document	Add document_level_security filter	Customer service reps see only their assigned tickets
Per-field	Add field_level_security masked_fields	Mask customer PII fields (email, phone) from analytics team
DLS + FLS combined	Apply both document and field rules to one role	Admin role: all docs, all fields. Analyst role: masked PII, active

12-22

OpenSearch Performance Tuning

Most OpenSearch performance problems trace to one of three causes: JVM pressure (too much data per node), slow queries (missing filters, too many facets), or shard imbalance (too many small shards). Here are the ShopFlow-specific tuning settings.

Tuning Area	Problem Sign	Fix	ShopFlow Setting
JVM heap	JVMMemoryPressure > 85%; frequent GC pauses	Increase heap size or add nodes; aim for 50% JVM heap utilization	50% JVM Heap Utilization
Shard sizing	Shards < 10 GB or > 50 GB; search hitting shard cap	Importing is halted 10-30 GB of data; delete empty indices	200 GB / 2 shards = 100 GB per shard
Query caching	Same filter query runs repeatedly; circuit breaker tripped	Block rather than rechecked automatically; ensure filter queries don't timeout	Use filter queries w/ doc counts and day/month filters
Bulk indexing	Slow ingestion throughput	Use bulk API (section 12-06); set refresh_interval=30s during bulk load	Search 30 seconds during bulk load (instead of 10s)
Request cache	Aggregation queries are slow	Aggregations with same filter are request-cached	Cache only queries size=0 to optimize hit

12-23

Black Friday OpenSearch Readiness

Search is in the critical path on Black Friday: every product page load triggers a search query. OpenSearch must handle 10x normal query rate with < 50ms P99 latency.

Checklist Item	Normal	Black Friday Action
Query rate	500 queries/second	5,000 queries/second — verify nodes can handle; add 2 replica shards (1000 shards)
JVM pressure	~50%	Pre-warm by running a search query load test; JVM compiles hot paths; add heap memory
Replica count	1 replica per primary	Increase to 2 replicas before Black Friday: more replicas = more parallelism
Request timeout	30 seconds (default)	Set client timeout to 3 seconds; return partial results on timeout rather than fail
Circuit breaker	OpenSearch built-in	Ensure requestCircuitBreaker threshold set in cluster settings; prevents OOM

12-24

Snapshots and Disaster Recovery

OpenSearch automated snapshots run daily and are stored in an S3 bucket managed by AWS. Manual snapshots let you create point-in-time backups before risky operations (reindex, mapping change) and restore to the same or a different domain.

■ Create and Restore Manual Snapshot

1. OpenSearch Dashboards -> Dev Tools
2. Register a snapshot repository (one-time setup):
3. PUT /_snapshot/shopflow-backups {"type":"s3","settings":{"bucket":"shopflow-opensearch-snapshots","region":"us-east-1"}}
4. Create manual snapshot before reindex:
5. PUT /_snapshot/shopflow-backups/pre-reindex-2024-11-28 {"indices":"products","include_global_state":false}
6. Check snapshot status: GET /_snapshot/shopflow-backups/pre-reindex-2024-11-28
7. Wait for state: "SUCCESS"
8. --- If reindex goes wrong ---
9. Restore: POST /_snapshot/shopflow-backups/pre-reindex-2024-11-28/_restore {"indices":"products","rename_pattern":"(.+)", "rename_replacement":"\$1-restored"}
10. Update alias to point to products-restored index
11. Verify document count matches pre-reindex count

Snapshot Type	Frequency	Retention	Use Case
Automated (AWS-managed)	Daily at domain creation hour	4 days	Recovery from accidental index deletion; baseline DR
Manual (S3-registered)	On-demand	Until you delete	Pre-risky-operation backup; cross-account restore; lon
Index snapshot (partial)	On-demand	Until you delete	Backup single index without affecting others

12-25

Amazon OpenSearch Ingestion — Zero-ETL Pipelines

OpenSearch Ingestion (powered by OpenSearch Data Prepper) is a managed pipeline that continuously streams data from DynamoDB, S3, RDS, and other sources directly into OpenSearch — without writing Lambda functions or custom ETL code.

```
# OpenSearch Ingestion pipeline: DynamoDB -> OpenSearch (zero-ETL) # Create via AWS Console ->
OpenSearch -> Ingestion -> Create pipeline # Or as CloudFormation/CDK (newer feature) version:
"2" dynamodb-pipeline: source: dynamodb: acknowledgments: true tables: - table_arn:
"arn:aws:dynamodb:us-east-1:123:table/shopflow-products" stream: start_position: "LATEST"
export: s3_bucket: "shopflow-opensearch-export" s3_prefix: "export/" aws: region: "us-east-1"
sts_role_arn: "arn:aws:iam::123:role/OpenSearchIngestionRole" sink: - opensearch: hosts: -
"https://search-shopflow-search-xyz.es.amazonaws.com" index: "products" index_type: "custom"
document_id: "${getMetadata("primary_key")}" aws: sts_role_arn:
"arn:aws:iam::123:role/OpenSearchIngestionRole" region: "us-east-1"
```

INFO Zero-ETL vs Lambda Approach

ShopFlow currently uses DynamoDB Streams -> SearchIndexLambda -> OpenSearch. OpenSearch Ingestion does the same without the Lambda: managed pipeline, automatic checkpointing, built-in retry, and filtering/transformation DSL. Migration from Lambda to Ingestion pipeline: create the pipeline, validate it matches Lambda output, then disable the Lambda trigger. Reduces infrastructure complexity and eliminates Lambda cold start latency.

12-26

OpenSearch Serverless — No Cluster Management

OpenSearch Serverless provides search and analytics without provisioning or managing an OpenSearch cluster. You define collections (groups of indices), and AWS automatically scales capacity based on demand. No nodes, no shard configuration, no JVM tuning.

Feature	OpenSearch Service (Managed)	OpenSearch Serverless	
Infrastructure	2+ data nodes you choose and size	Fully serverless — AWS manages capacity	
Minimum cost	~\$82/month (1 x r6g.large 24/7)	\$0 when idle + \$0.24/OCU-hr when active (min 2 OCUs/collection)	
Scaling	Manual (add nodes) or auto-scaling (UltraWarm)	Automatic — scales up instantly; scales to zero when idle	
Shard management	You configure shards, replicas, refresh intervals	Fully managed — no shard configuration	
Supported features	All OpenSearch features	Subset: search and time-series analytics; no ISM, cross-cluster, or alerting	
Best for	Production with steady traffic; need all features	Dev/test, sporadic workloads, variable traffic	
ShopFlow recommendation	shopflow-search (production) — Managed	shopflow-search-dev (development) — Serverless to save costs	

12-27

OpenSearch Cost Analysis — ShopFlow Production

OpenSearch Service cost has three components: data node compute, EBS storage, and UltraWarm. Right-sizing these significantly impacts the bill.

Cost Component	Config	Monthly Cost	Notes
Data nodes (hot)	2 x r6g.large.search	\$175	\$0.12/hr x 2 nodes x 730 hrs. Graviton saves 20%
EBS storage (hot)	2 x 100 GB gp3	\$24	\$0.122/GB-month x 200 GB total
UltraWarm nodes	2 x ultrawarm1.medium.search	\$96	\$0.065/hr x 2 x 730 hrs
UltraWarm storage	500 GB (30-90 day logs)	\$12	\$0.024/GB-month x 500 GB
Automated snapshots	Included	\$0	AWS stores 14 automated snapshots in managed S3
Manual snapshots	50 GB in self-managed S3	\$1.15	\$0.023/GB-month S3 Standard
TOTAL	—	\$308/month	Cheaper than self-managed EC2+EBS (\$450+ for dev)

TIP Dev/Test Cost Savings

Run dev/test OpenSearch on a t3.small.search single-node domain (\$0.036/hr = \$26/month) instead of the production r6g.large configuration. For very low-volume dev, use OpenSearch Serverless: no data during working hours = ~\$0. Delete the dev domain on Friday, recreate Monday — all indices reconstructable from CDK.

12-28

OpenSearch Alerting and Anomaly Detection

OpenSearch has built-in alerting (trigger notifications when a query result meets a condition) and anomaly detection (ML-powered detection of unusual patterns in time-series data). ShopFlow uses both for search quality monitoring.

Feature	How It Works	ShopFlow Use Case
OpenSearch Alerting	Define a monitor: an OpenSearch query that runs on a schedule; if the results meet a condition, trigger a notification	Alert when search results are >5% fewer than expected, or when logs index is over 100GB
Anomaly Detection	Train a model on time-series data (search latency, indexing rate); model detects spikes or dips	Detect indexing rate anomalies during Black Friday when demand is high
Document-level Monitoring	Query returns matching documents; alert on each document	Alert on each error logged to OpenSearch with severity=CRITICAL

12-29

OpenSearch Quick Reference — Key API Operations

Essential OpenSearch API commands. Run these from OpenSearch Dashboards Dev Tools or via curl with SigV4 signing from the AWS CLI.

Operation	API Call
Create index	PUT /products {"settings":{"..."},"mappings":{"..."}}

Operation	API Call
Index a document	PUT /products/_doc/SKU-12345 {"name":"Nike Air Max","price":149.99,...}
Bulk index	POST /products/_bulk (then NDJSON lines of action + source)
Search query	POST /products/_search {"query":{"..."},"aggs":{"..."},"size":20}
Get document by ID	GET /products/_doc/SKU-12345
Count documents	GET /products/_count {"query":{"match_all":{"...}}}
Cluster health	GET /_cluster/health?v
List indices	GET /_cat/indices?v&h=index,health,docs.count,store.size
Check shard allocation	GET /_cat/shards?v&h=index,shard,prirep,state,node
Update alias	POST /_aliases {"actions":[{"add":{"index":"products-v2","alias":"products"}},{"remove":{"index":"products-v1","alias":"products"}}]}
Reindex	POST /_reindex {"source":{"index":"products-v1"},"dest":{"index":"products-v2"}}
Delete index	DELETE /products-v1 (careful — irreversible)

12-30

Search Analytics — Understanding What Users Search For

Every search query is a signal: what are customers looking for? What do they not find? ShopFlow logs all search queries and result metrics to an OpenSearch analytics index for weekly review and search quality improvement.

```
// Log search events for analytics (Java Lambda) SearchAnalyticsEvent event =
SearchAnalyticsEvent.builder() .timestamp(Instant.now()) .sessionId(request.sessionId())
.query(request.query()) // "red running shoes" .filters(request.filters()) // {brand: Nike,
price_max: 100} .resultCount(response.total().value()) // 47 results
.resultIds(topResultIds(response, 5)) // top 5 product IDs shown .searchDurationMs(durationMs)
// 42ms .zeroResults(response.total().value() == 0) .build(); // Index to analytics-specific
index (not products index) client.index(IndexRequest.of(req -> req .index("search-analytics-" +
LocalDate.now()) // daily index for easy rollover .document(event));
```

Analytics Insight	OpenSearch Query	Action Taken by ShopFlow
Top 20 zero-result queries	Aggregate queries where resultCount=0, sort by frequency	Add synonyms for common misspellings; add missing product categories
Most popular searches	Aggregate query field, sort by doc_count	Ensure top 100 queries return relevant results; A/B test improvements
Average search latency by hour	Date histogram on timestamp, avg of searchDurationMs	Identify peak hours; optimize caching for busiest periods
Filter usage patterns	Aggregate which filters users combine most frequently	Highlight most-used filter combinations in UI for faster product discovery

12-31

Best Practices and Anti-Patterns — OpenSearch

Best Practice	Anti-Pattern to Avoid
Use index aliases; never hardcode index names in application code	Direct index name — forces code deploy for every reindex or schema change
Use keyword type for brand, category, productId (exact match for filters)	text type for filters — tokenized; exact filter "Nike" misses documents
Set number_of_shards at creation based on data size (Default: 1 shard per 10 GB)	Default 1 shard for 100 GB index — single overloaded shard; no scale-out possible
Enable automated snapshots; manual snapshot before reindex	No snapshots — if reindex corrupts data, no rollback
Monitor JVM pressure; alarm at 85%; scale before hitting 90%	Ignoring JVM metrics until cluster turns red from OOM
Use date math (now/d) in filters so results are cached; Ex: now-1d to 1d	Exact now timestamp filter — every query is unique, no cache hits
Implement search analytics logging; review zero-result queries weekly	No search analytics — flying blind on search quality; customers leave silently
Use ISM policy to auto-move old indices to UltraWarm and delete after 1 year	Indices never deleted — storage fills, indexing stops, cluster turns red

Three common search requirements that require specific OpenSearch features: nested objects (product variants), search result highlighting, and deep pagination.

```
// 1. Highlighting – show WHY a result matched SearchRequest.of(req -> req .query(q -> q.match(m
-> m.field("description").query("waterproof"))) .highlight(h -> h .fields(Map.of(
"description", HighlightField.of(hf -> hf .numberOfFragments(2) .fragmentSize(150))))
.preTags("[MATCH]") // wrap matched terms in a tag .postTags("[/MATCH]"))) // Result: "...with
[MATCH]waterproof[/MATCH] membrane and sealed seams..." // 2. Search After – efficient deep
pagination (better than from/size for > 10K results) // First page: sort by _score desc, then
productId (tie-breaker must be unique) SearchRequest firstPage = SearchRequest.of(req -> req
.sort(List.of( SortOptions.of(so -> so.score(ss -> ss.order(SortOrder.Desc))),
SortOptions.of(so -> so.field(f -> f.field("productId").order(SortOrder.Asc)))) .size(20)); //
Use last hit's sort values as cursor for next page List[FieldValue] cursor = lastHit.sort(); //
[score, "SKU-99999"] SearchRequest nextPage = SearchRequest.of(req -> req
.searchAfter(cursor).size(20)); // efficient: no deep offset scan // 3. Nested objects –
product variants (size, color) as nested documents // Requires "nested" field type in mapping +
nested query SearchRequest.of(req -> req .query(q -> q .nested(n -> n .path("variants")
.query(vq -> vq .bool(b -> b .must(m -> m.term(t -> t.field("variants.size").value("10")))
.must(m -> m.term(t -> t.field("variants.color").value("red"))) .must(m -> m.range(r ->
r.field("variants.stock").gt(JsonData.of(0))))))))))
```

ShopFlow serves customers globally. A Spanish-speaking user searching "zapatos rojos" should find "red shoes" even if product names are only in English. Multi-language search combines language-specific analyzers with synonym dictionaries and machine translation.

Language Feature	Implementation	ShopFlow Status
Language-specific analyzers	Separate field per language: name_en (English stemmer), name_es (Spanish stemmer), name_de (German stemmer)	Planned
Synonym file	OpenSearch synonym_graph filter: "trainers => sneakers, jumpers => sweaters"	Active
Machine translation	Translate user query from detected language to English before search using Google Translate API	Planned
Multi-match across languages	If multilingual product names indexed: query against name_en AND name_es AND name_de with multi_match query	Planned

TIP Quick Win: Synonym Dictionary

The highest-ROI search improvement is a synonym dictionary. ShopFlow added 200 synonym pairs in one day (trainer=sneaker, trousers=pants, jumper=sweater for UK English) and saw zero-result rate drop from 8% to 3%. Create the synonym file as an S3 object; reference in the index analyzer configuration. Update the file and call the reload synonyms API without reindexing.

OpenSearch Alerting lets you define monitors that run saved queries on a schedule and send notifications when thresholds are exceeded. ShopFlow uses this to alert when the zero-result search rate rises above 5% — indicating a catalog gap or search quality issue.

■ Create Zero-Result Rate Alert

1. OpenSearch Dashboards -> Alerting -> Monitors -> Create monitor
2. Monitor name: shopflow-zero-result-rate
3. Monitor type: Per query monitor
4. Data source: Index = search-analytics-* (all daily analytics indices)
5. Time field: timestamp | Frequency: Every 1 hour
6. Query: Count of documents where resultCount=0, divided by total count in last 1 hour
7. Trigger: if zero_result_pct > 0.05 (5%) -> send notification
8. Actions: Destination = SNS topic shopflow-ops-alerts
9. Message: "Zero-result rate is {{ctx.results.0.aggregations.zero_result_pct.value}}% in the last hour"
10. Click Create -> monitor starts running immediately
11. Test: manually index a search_analytics document with resultCount=0; verify alert fires

Alert Type	Query	Threshold	Notification
Zero-result rate	% of searches returning 0 hits in last hour	> 5%	SNS -> ops-alerts Slack channel
Search P99 latency	99th percentile of searchDurationMs in last 500ms	> 500ms	SNS -> on-call PagerDuty
Index doc rate drop	Documents indexed per hour drops vs 24h 50% average	< 50%	SNS -> ops-alerts email
Cluster yellow/red	ClusterStatus metric from CloudWatch (not OpenSearch)	Any OpSts Search	CloudWatch Alarm -> SNS -> PagerDuty P